

Minicurso Prático

Git & GitHub na **Prática**

Dominando o Fluxo de Trabalho Profissional em Equipes Reais

Por que estamos aqui?

- ✗ **O mito do add/commit/push:** Saber os comandos básicos não te prepara para o mercado.
- ✗ **Trabalho em Equipe:** Em empresas reais, dezenas de pessoas editam a mesma base de código simultaneamente.
- ✓ **Nosso Objetivo Hoje:** Simular um ambiente corporativo real. Vamos construir, colaborar, revisar e resolver problemas juntos.

600 × 400

O Projeto: **Dev Directory**

Construiremos um diretório de perfis usando HTML/CSS/JS. Simples no código, complexo na colaboração.

Nossa Jornada Hoje (180 min)

01 Planejamento

Configuração do ambiente e gestão visual de tarefas com o GitHub Projects (Kanban).

02 Isolamento

Criação de Branches para desenvolver features sem quebrar o sistema principal (a branch main).

03 Integração

Abertura de Pull Requests (PRs), Code Review da equipe e validação por CI (GitHub Actions).

04 Caos Controlado

Simulação de conflitos reais de merge e uso de Fetch, Rebase e Cherry-Pick para resolução.

Bloco 1: Gestão com GitHub Projects

- ☑ **Cultura do Planejamento:** Ninguém escreve código sem uma *Issue* definida.
- 📋 **Kanban na Prática:** Colunas Todo, In Progress e Done dão visibilidade ao trabalho da equipe.
- 👤 **Assignment:** Vamos criar Issues para cada um e nos atribuir às tarefas antes de codar.

600 × 400

Bloco 2: O poder das Branches



A Regra de Ouro

Nunca (jamais) commite diretamente na branch ``main``. A ``main`` é o que está em produção.



Ambiente Seguro

Branches criam uma cópia do projeto onde você pode experimentar e quebrar coisas sem afetar ninguém.



Nomenclatura

Pratique padrões reais: ``feature/nome-aluno`` ou ``issue-12-add-profile``.

Bloco 3: Pull Requests e Code Review

-  **O Pedido Formal:** Um PR é um pedido para integrar seu código (sua branch) na `main`.
-  **Code Review:** Vamos revisar o código uns dos outros. Aprendizado acontece discutindo soluções.
-  **Continuous Integration (CI):** O GitHub Actions vai rodar automaticamente para validar se você não quebrou nada antes do merge.

600 × 400

Bloco 4: Sincronização Limpa (Fetch & Rebase)



1. Fetch

``git fetch origin``: Baixa as atualizações do GitHub sem tentar mesclar no seu código atual. É um "olhar sem tocar".



2. O Problema da Base

Sua branch foi criada ontem, mas a ``main`` já evoluiu hoje. Sua base está desatualizada.



3. Rebase

``git rebase origin/main``: Pega seus commits, atualiza a base da sua branch, e reaplica seus commits no topo. Mantém a história linear.

Bloco 4: Cherry-Pick e Resolução de Conflitos

-  **Cherry-Pick:** O transplante cirúrgico. Como pegar apenas UM commit específico (ex: um hotfix) e movê-lo para outra branch usando o hash.
-  **O Caos Programado:** Vamos criar um cenário onde dois alunos alteram a mesma linha de código.
-  **Resolvendo:** Lendo as marcações `<<<<<<< HEAD` e negociando qual código sobrevive ao merge.

600 × 400



A teoria acabou. É hora de acessar o repositório,
escolher sua Issue e começar a codar.

URL do Repositório na tela.